ONEDRAFT

CAD-AI — Aria Powered

Executable Application Stack & Implementation Architecture

Version 1.0

Status: Implementation Specification
Architecture: Model-Centric / Service-Oriented
Frontend: Next.js / React
Backend: Python / FastAPI
Database: PostgreSQL
Object Storage: S3-Compatible
Async Infrastructure: Redis + Worker Services
AI Layer: Aria Orchestration
Geometry Layer: Open CASCADE / Equivalent Geometry Kernel
Documentation: Vector CAD / ARCH D / PDF / DXF
Containerization: Docker
API Version: `/api/v1`

---

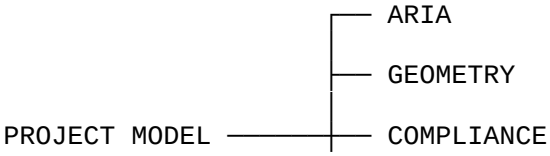
1. IMPLEMENTATION OBJECTIVE

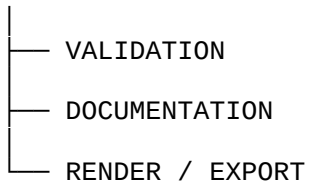
OneDraft now moves from architectural specification into executable software construction.

The implementation shall establish the first operational system capable of:

```
USER
 ↓
WEB APPLICATION
 ↓
API
 ↓
DOMAIN SERVICES
 ↓
PROJECT MODEL
 ↓
POSTGRESQL
```

with asynchronous computational services operating alongside the transactional core:





The Project Model remains the authoritative source of truth.

---

2. IMPLEMENTATION PRINCIPLE

OneDraft shall not allow the user interface, AI provider, geometry engine, drawing renderer, or document generator to become the system of record.

The authoritative sequence remains:

```
PROJECT
 ↓
PROJECT MODEL
 ↓
MODEL VERSION
 ↓
REVISION
 ↓
VALIDATION
 ↓
DOCUMENTATION
 ↓
ACCEPTANCE
```

Every derived artifact must identify the model state from which it originated.

---

3. TECHNOLOGY STACK

The initial implementation stack shall be:

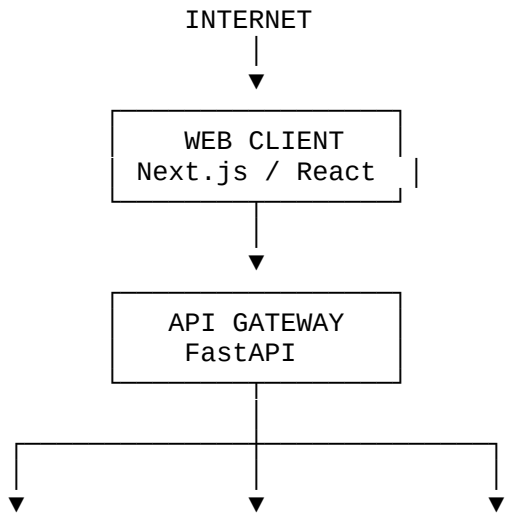
```
Frontend
    Next.js
    React
    TypeScript

API
    Python
    FastAPI
    Pydantic

Persistence
    PostgreSQL
    SQL migrations

Caching / Jobs
```

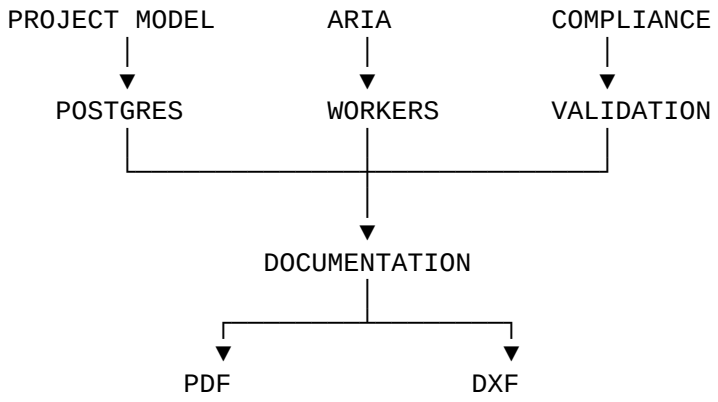
```
    Redis

Workers
    Python worker services

Object Storage
    S3-compatible storage

Geometry
    Open CASCADE or equivalent geometry kernel

Documentation
    Vector drawing pipeline
    ARCH D sheet generation
    PDF generation
    DXF export

AI
    Aria orchestration layer
    Provider adapter architecture

Infrastructure
    Docker
    Docker Compose for development

Testing
    Pytest
    API integration tests
    Geometry tests
    Golden-project regression tests

API Contract
    OpenAPI 3
```

---

4. APPLICATION TOPOLOGY

The initial runtime topology shall be:





---

5. REPOSITORY ROOT

The implementation repository shall use:

```
onedraft/
```

The repository structure shall be:

```
onedraft/
├── README.md
├── LICENSE
├── pyproject.toml
├── package.json
├── .env.example
├── docker-compose.yml
│
├── apps/
│   ├── web/
│   └── api/
│
├── services/
│   ├── aria/
│   ├── project_model/
│   ├── geometry/
│   ├── compliance/
│   ├── validation/
│   ├── documentation/
│   ├── rendering/
│   └── exports/
│
├── workers/
│   ├── ai/
│   ├── geometry/
│   ├── validation/
│   ├── drawing/
│   ├── document/
│   └── render/
│
├── database/
│   ├── migrations/
│   ├── seed/
```

```
│   └── schemas/
│
├── packages/
│   ├── domain/
│   ├── schemas/
│   ├── ui/
│   ├── geometry_types/
│   └── api_client/
│
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── geometry/
│   ├── ai/
│   ├── drawing/
│   ├── regression/
│   └── golden_projects/
│
├── infra/
│   ├── docker/
│   ├── deployment/
│   ├── monitoring/
│   └── security/
│
└── docs/
    ├── architecture/
    ├── api/
    ├── model/
    ├── compliance/
    └── operations/
```

---

6. FRONTEND APPLICATION

The web application shall reside at:

```
apps/web/
```

The frontend shall use:

```
Next.js
React
TypeScript
```

The frontend is responsible for:

```
authentication
project navigation
project specification
model inspection
drawing inspection
redline creation
validation results
document review
customer acceptance
```

The frontend shall not contain authoritative business logic.

---

7. FRONTEND ARCHITECTURE

The frontend shall be organized approximately as:

```
apps/web/
├── app/
├── components/
├── features/
│   ├── projects/
│   ├── model/
│   ├── drawings/
│   ├── redlines/
│   ├── compliance/
│   ├── validation/
│   └── documents/
├── lib/
├── hooks/
├── state/
├── styles/
└── public/
```

The UI consumes the API contract.

It does not directly access:

```
PostgreSQL
Redis
object storage
geometry workers
AI providers
```

---

8. API APPLICATION

The backend shall reside at:

```
apps/api/
```

The API framework shall be:

```
FastAPI
```

The API shall expose:

```
/api/v1
```

The initial application structure shall be:

```
apps/api/
```

```
├── main.py
├── dependencies.py
├── middleware/
├── routes/
├── auth/
├── services/
├── schemas/
├── errors/
└── configuration/
```

---

9. API ROUTING

The root application shall register:

```
/api/v1/projects
/api/v1/organizations
/api/v1/requirements
/api/v1/buildings
/api/v1/levels
/api/v1/spaces
/api/v1/elements
/api/v1/constraints
/api/v1/commands
/api/v1/revisions
/api/v1/redlines
/api/v1/views
/api/v1/drawings
/api/v1/sheets
/api/v1/documents
/api/v1/compliance
/api/v1/validation
/api/v1/exports
/api/v1/jobs
/api/v1/acceptance
/api/v1/events
```

---

10. DOMAIN LAYER

The domain layer shall reside in:

```
packages/domain/
```

The domain layer defines the semantic model independently from HTTP.

Initial domain concepts include:

```
Organization
User
Project
Requirement
Site
```

```
Building
Level
Space
Element
Assembly
Constraint
Relationship
ModelVersion
Revision
ChangeRecord
AriaCommand
DesignProposal
View
Drawing
Sheet
Redline
ValidationRun
ValidationResult
CodeManifest
DocumentPackage
AcceptanceRecord
AuditEvent
```

The domain layer shall not depend on FastAPI.

---

11. PYDANTIC CONTRACT LAYER

Machine-facing schemas shall reside in:

```
packages/schemas/
```

These models shall define:

```
request models
response models
command models
event models
job models
validation models
drawing models
document models
```

Pydantic models shall enforce:

```
type validation
required fields
enumerated values
nested structure
serialization
API contract compatibility
```

---

12. API CLIENT

The frontend shall consume a generated or centrally maintained TypeScript API client:

`packages/api_client/`

The client shall be generated from:

`docs/openapi/openapi.yaml`

The frontend should not manually duplicate API endpoint definitions.

---

13. PROJECT MODEL SERVICE

The Project Model service shall reside at:

`services/project_model/`

This service is the primary application-domain boundary.

Its responsibility is to:

```
load project state
validate commands
apply model mutations
preserve constraints
create model versions
create revisions
record changes
emit events
```

The service shall never permit arbitrary mutation of the project model.

---

14. PROJECT MODEL COMMAND PIPELINE

Every model mutation shall follow:

```
REQUEST
 ↓
AUTHENTICATION
 ↓
AUTHORIZATION
 ↓
LOAD CURRENT REVISION
 ↓
VERIFY EXPECTED REVISION
 ↓
```

```
VALIDATE COMMAND
 ↓
LOAD CONSTRAINTS
 ↓
RESOLVE TARGET OBJECTS
 ↓
CALCULATE MUTATION
 ↓
APPLY MUTATION
 ↓
VALIDATE RESULT
 ↓
CREATE MODEL VERSION
 ↓
CREATE REVISION
 ↓
RECORD CHANGE
 ↓
RECORD EVENT
 ↓
COMMIT
```

No step may silently bypass the transactional model boundary.

---

15. COMMAND SERVICE

The initial command service shall implement:

```
CREATE_OBJECT
UPDATE_OBJECT
DELETE_OBJECT
MOVE_ELEMENT
ROTATE_ELEMENT
RESIZE_ELEMENT
CHANGE_PARAMETER
CREATE_RELATIONSHIP
DELETE_RELATIONSHIP
APPLY_REDLINE
```

Additional operations shall be added without altering the fundamental command boundary.

---

16. ARIA SERVICE

The Aria service shall reside at:

```
services/aria/
```

Aria is the intelligence and orchestration layer.

Aria shall interpret user intent into structured OneDraft commands.

The fundamental pipeline is:

```
USER LANGUAGE
 ↓
ARIA INTERPRETATION
 ↓
STRUCTURED COMMAND
 ↓
PROJECT MODEL VALIDATION
 ↓
COMMAND EXECUTION
 ↓
VALIDATION
 ↓
REVISION
```

Aria does not directly mutate PostgreSQL.

---

17. ARIA PROVIDER ABSTRACTION

The underlying AI provider shall remain abstracted.

The architecture shall be:

```
Aria
 ↓
Aria Orchestrator
 ↓
Provider Adapter
 ↓
AI Model Provider
```

The application must not bind the public OneDraft API to a particular underlying model vendor.

---

18. ARIA COMMAND CONTRACT

The minimum command representation shall include:

```
command_id
project_id
revision_id
operation
target_objects
parameters
preserve_constraints
validation_requirements
reason
confidence
requested_by
```

The command must be deterministic enough for the Project Model service to validate it.

---

19. ARIA SAFETY BOUNDARY

Aria shall not have direct authority to perform:

```
SQL execution
arbitrary database writes
filesystem replacement
production configuration changes
permission modification
billing modification
```

Aria operates through application capabilities.

---

20. GEOMETRY SERVICE

The geometry service shall reside at:

```
services/geometry/
```

The geometry subsystem shall provide:

```
2D geometry
3D geometry
topology
boolean operations
intersections
offsets
extrusions
transformations
bounding boxes
geometry validation
geometry serialization
```

The preferred computational foundation is:

```
Open CASCADE
```

or an equivalent geometry kernel where implementation requirements dictate.

---

21. GEOMETRY ABSTRACTION

OneDraft shall not expose raw geometry-kernel objects throughout the application.

The system shall use an internal geometry abstraction:

```
GeometryReference
GeometryPrimitive
GeometryTransform
GeometryOperation
GeometryResult
```

This prevents the geometry engine from becoming inseparable from the domain model.

---

22. GEOMETRY WORKER

Expensive geometry operations execute asynchronously.

The worker shall receive:

```
project_id
model_version_id
element_id
geometry_version
operation
input_geometry
parameters
```

The worker returns:

```
geometry_reference
geometry_hash
bounding_box
geometry_version
status
```

---

23. GEOMETRY STALENESS CONTROL

A geometry result is valid only when:

```
project_id
+
model_version_id
+
element_id
+
geometry_version
```

match the current expected state.

A stale geometry result shall never overwrite current geometry.

---

24. COMPLIANCE SERVICE

The compliance service shall reside at:

```
services/compliance/
```

Its responsibility is to transform jurisdictional source material into machine-evaluable project constraints.

The pipeline is:

```
JURISDICTION
 ↓
CODE SOURCES
 ↓
CODE RULES
 ↓
CODE MANIFEST
 ↓
PROJECT APPLICABILITY
 ↓
CONSTRAINTS
 ↓
VALIDATION
```

---

25. COMPLIANCE PROVENANCE

Every regulatory decision shall maintain provenance through:

```
code_source_id
rule_id
citation
source_version
effective_date
retrieval_timestamp
content_hash
```

A final project revision shall identify the exact regulatory manifest used during validation.

---

26. VALIDATION SERVICE

The validation service shall reside at:

```
services/validation/
```

Initial validation domains:

```
CODE
SPATIAL
```

```
ACCESSIBILITY
DOCUMENT
COORDINATION
MODEL_INTEGRITY
GEOMETRY
```

Validation produces immutable validation results associated with the specific project revision.

---

27. VALIDATION PIPELINE

```
MODEL VERSION
 ↓
CODE MANIFEST
 ↓
VALIDATION RULE SET
 ↓
VALIDATION ENGINE
 ↓
VALIDATION RESULTS
 ↓
REVISION STATUS
```

A validation result shall never be detached from the model state against which it was generated.

---

28. DOCUMENTATION SERVICE

The documentation service shall reside at:

```
services/documentation/
```

Its responsibility is to derive architectural documentation from the Project Model.

It shall generate:

```
plans
elevations
sections
details
schedules
notes
dimensions
annotations
sheet compositions
title blocks
revision blocks
```

---

29. DRAWING DERIVATION

Drawings are derived views.

The relationship is:

```
PROJECT MODEL
 ↓
MODEL VERSION
 ↓
VIEW
 ↓
DRAWING
 ↓
SHEET
```

The drawing itself is not the authoritative building model.

---

30. ARCH D OUTPUT

The documentation system shall support:

```
ARCH D
```

as an initial sheet format.

The sheet system shall preserve:

```
paper dimensions
drawing scale
title block
sheet number
drawing number
revision
annotations
graphic standards
viewports
```

---

31. VECTOR-FIRST DOCUMENTATION

The drawing pipeline shall be vector-oriented.

The preferred output hierarchy is:

```
PROJECT MODEL
 ↓
VECTOR DRAWING MODEL
 ↓
DXF / CAD REPRESENTATION
 ↓
```

```
PDF
```

Raster imagery may be included where necessary but shall not become the primary architectural representation.

---

32. RENDERING SERVICE

The rendering service shall reside at:

```
services/rendering/
```

Rendering shall provide:

```
drawing previews
sheet previews
model previews
PDF rendering
review images
```

Rendering is derived from the documentation state.

---

33. EXPORT SERVICE

The export service shall reside at:

```
services/exports/
```

Initial export formats:

```
PDF
DXF
```

Future formats may be added through adapters.

---

34. DOCUMENT PACKAGE PIPELINE

The final document pipeline shall be:

```
PROJECT
  ↓
MODEL VERSION
  ↓
REVISION
  ↓
CODE MANIFEST
  ↓
```

```
VALIDATION
 ↓
DRAWING GENERATION
 ↓
SHEET COMPOSITION
 ↓
DOCUMENT PACKAGE
 ↓
HASH
 ↓
CUSTOMER ACCEPTANCE
```

---

35. OBJECT STORAGE

Large and derived artifacts shall not be stored directly inside PostgreSQL.

Object storage shall contain:

```
geometry payloads
uploaded project documents
source regulatory artifacts
drawing files
DXF files
PDF files
rendered previews
document packages
```

The database stores references and cryptographic hashes.

---

36. OBJECT STORAGE INTERFACE

The application shall interact with object storage through an abstraction:

```
ObjectStore
```

Initial operations:

```
put()
get()
delete()
exists()
presign()
hash()
```

The implementation shall support S3-compatible storage.

---

37. REDIS

Redis shall provide the initial asynchronous coordination layer.

Primary uses:

```
job queues
worker coordination
short-lived cache
job status
distributed locks where required
```

Redis shall not become the authoritative source of project state.

---

38. WORKER ARCHITECTURE

Workers shall reside under:

```
workers/
```

Initial workers:

```
workers/ai/
workers/geometry/
workers/validation/
workers/drawing/
workers/document/
workers/render/
```

Each worker consumes a defined job contract.

---

39. JOB CONTRACT

Every asynchronous job shall contain:

```
job_id
project_id
job_type
requested_by
model_version_id
revision_id
payload
created_at
status
attempt
```

Where applicable:

```
code_manifest_id
```

```
drawing_id
view_id
document_package_id
```

---

40. JOB STATE MACHINE

The common job lifecycle shall be:

```
QUEUED
 ↓
RUNNING
 ↓
VALIDATING
 ↓
COMPLETED
```

Failure states:

```
FAILED
CANCELLED
```

Retries shall be controlled and idempotent.

---

41. IDEMPOTENT WORKERS

Workers shall not blindly duplicate output.

Before committing a result, a worker verifies:

```
job identity
model version
revision
input hash
output existence
```

A retry of the same job must produce the same logical result or safely recognize that the result already exists.

---

42. DATABASE ACCESS

Application services shall access PostgreSQL through controlled repositories or domain persistence interfaces.

Example:

```
ProjectRepository
```

```
RevisionRepository
ElementRepository
ConstraintRepository
CommandRepository
DrawingRepository
ValidationRepository
```

Direct SQL from frontend or AI components is prohibited.

---

43. TRANSACTION SERVICE

The Project Model transaction boundary shall provide:

```
begin
verify
mutate
record
version
commit
rollback
```

The service shall guarantee atomicity for model-state mutations.

---

44. OPTIMISTIC CONCURRENCY

Commands shall include an expected revision.

Example:

```
expected_revision = 14
```

The server compares:

```
expected_revision
```

against:

```
current_revision
```

If they differ, the mutation is rejected rather than silently applied to newer state.

---

45. REVISION CREATION

Successful model mutations shall create a new revision when the operation constitutes a project-state change.

Example:

```
Revision 14
    ↓
ARIA COMMAND
    ↓
Revision 15
```

The previous revision remains immutable.

---

46. EVENT CREATION

Successful mutations shall produce an audit event.

Example:

```
MODEL_ELEMENT_MOVED
```

with:

```
actor
project
revision
object
before_state
after_state
command_id
timestamp
```

The event record is append-only.

---

47. DATABASE MIGRATION IMPLEMENTATION

The initial migration package shall be:

```
database/migrations/
```

The first migration shall establish:

```
extensions
schemas
identity
projects
model
geometry
documentation
compliance
validation
```

```
revisions
audit
billing
system
```

Subsequent migrations shall follow the migration order established by the Database Schema & OpenAPI Contract.

---

48. FIRST MIGRATION

The first executable database artifact shall be:

```
database/migrations/001_initial_schema.sql
```

It shall establish the initial PostgreSQL foundation required by the OneDraft domain model.

---

49. OPENAPI IMPLEMENTATION

The machine-readable API contract shall reside at:

```
docs/openapi/openapi.yaml
```

The specification shall define:

```
paths
operations
parameters
request schemas
response schemas
error schemas
security schemes
job schemas
pagination
```

The OpenAPI specification is the contract between services and clients.

---

50. DOMAIN TYPES IMPLEMENTATION

The Python domain types shall reside under:

```
packages/domain/
```

and shall include:

```
project.py
building.py
```

```
level.py
space.py
element.py
assembly.py
constraint.py
relationship.py
revision.py
command.py
drawing.py
redline.py
validation.py
document.py
acceptance.py
```

Shared types shall be centralized to prevent incompatible representations across services.

---

51. PROJECT MODEL SERVICE IMPLEMENTATION

The first executable service shall be:

```
services/project_model/project_model_service.py
```

It shall expose application-level operations such as:

```
create_project()
get_project()
create_element()
update_element()
execute_command()
create_revision()
get_revision()
reconstruct_revision()
```

---

52. COMMAND EXECUTION INTERFACE

Conceptually:

```
execute_command(
    project_id,
    command,
    expected_revision
)
```

The method shall:

```
load project
load revision
authorize request
```

```
validate command
resolve targets
evaluate constraints
apply mutation
validate resulting model
create model version
create revision
record changes
record event
commit
```

---

53. MODEL RECONSTRUCTION

The Project Model service shall provide:

```
reconstruct_revision(project_id, revision_id)
```

The reconstruction process shall:

```
load baseline
load revision chain
apply immutable change records
resolve object relationships
resolve geometry references
return reconstructed model state
```

This becomes a foundational integrity test.

---

54. PROJECT SNAPSHOT

A model version shall contain or reference sufficient state to reconstruct the project.

The implementation may use:

```
snapshot + change records
```

rather than requiring every historical state to be rebuilt from the original project indefinitely.

---

55. MODEL HASHING

Each model version shall receive:

```
state_hash
```

The hash shall represent the canonicalized model state.

Equivalent canonical states must generate deterministic hashes.

---

56. GEOMETRY HASHING

Every stored geometry artifact shall receive:

```
geometry_hash
```

The geometry hash provides:

```
deduplication
integrity verification
staleness detection
reproducibility
```

---

57. DOCUMENT HASHING

Every finalized document package shall receive:

```
document_hash
```

The package hash shall be calculated after the package is completely generated.

The acceptance record shall identify the package hash through its associated document package.

---

58. GOLDEN PROJECT

The first complete OneDraft regression project shall be a controlled test project.

It shall contain:

```
site
building
multiple levels
rooms
walls
doors
windows
basic assemblies
dimensions
plans
elevations
sections
ARCH D sheets
```

The golden project becomes the canonical integration fixture.

59. FIRST END-TO-END TEST

The first end-to-end automated test shall execute:

```
CREATE ORGANIZATION
 ↓
CREATE USER
 ↓
CREATE PROJECT
 ↓
CREATE REQUIREMENTS
 ↓
CREATE BUILDING
 ↓
CREATE LEVELS
 ↓
CREATE SPACES
 ↓
CREATE WALLS
 ↓
CREATE DOORS
 ↓
CREATE WINDOWS
 ↓
GENERATE DRAWINGS
 ↓
SUBMIT ARIA COMMAND
 ↓
VALIDATE COMMAND
 ↓
CREATE REVISION
 ↓
REGENERATE DRAWINGS
 ↓
CREATE REDLINE
 ↓
RESOLVE REDLINE
 ↓
RUN VALIDATION
 ↓
GENERATE DOCUMENT PACKAGE
 ↓
HASH PACKAGE
 ↓
RECORD ACCEPTANCE
 ↓
RECONSTRUCT PROJECT HISTORY
```

---

60. TESTING LAYERS

Testing shall be separated into:

```
unit
integration
API
database
geometry
AI command interpretation
drawing
documentation
regression
golden project
```

Each layer validates a distinct architectural boundary.

---

61. UNIT TESTS

Unit tests shall verify:

```
domain rules
command validation
constraint evaluation
revision numbering
hash generation
pagination
schema validation
state transitions
```

---

62. API TESTS

API tests shall verify:

```
authentication
authorization
request validation
response schemas
error handling
idempotency
revision conflicts
job creation
resource access
```

---

63. GEOMETRY TESTS

Geometry tests shall verify:

```
wall creation
wall movement
wall intersection
```

```
door insertion
window insertion
space boundaries
level geometry
section generation
elevation generation
```

Geometry tests shall include numerical tolerances appropriate to the geometry engine.

---

64. DRAWING TESTS

Drawing tests shall verify:

```
correct model version
correct revision
correct view
correct scale
correct dimensions
correct annotations
correct sheet placement
correct ARCH D output
```

---

65. GOLDEN DRAWING TESTS

Golden projects shall produce deterministic reference artifacts.

Regression comparison shall verify:

```
geometry
object counts
sheet structure
drawing identifiers
revision identifiers
document hashes
```

Visual comparison may supplement structural comparison.

---

66. ERROR HANDLING

The API shall expose standardized errors.

Initial error classes:

```
AUTHENTICATION_ERROR
AUTHORIZATION_ERROR
NOT_FOUND
VALIDATION_ERROR
```

```
REVISION_CONFLICT
CONSTRAINT_CONFLICT
MODEL_CONFLICT
GEOMETRY_ERROR
JOB_ERROR
DOCUMENT_GENERATION_ERROR
INTERNAL_ERROR
```

Internal implementation details shall not be exposed unnecessarily through API responses.

---

67. OBSERVABILITY

The implementation shall include:

```
structured logging
request IDs
job IDs
project IDs
revision IDs
model version IDs
metrics
health checks
```

Critical operations shall be traceable across:

```
API
service
worker
database
object storage
```

---

68. HEALTH ENDPOINTS

The API shall expose:

```
GET /health
GET /ready
```

Health shall identify application availability.

Readiness shall verify required dependencies such as:

```
PostgreSQL
Redis
object storage
```

where appropriate.

---

69. CONFIGURATION

Environment-specific configuration shall be externalized.

Example:

```
DATABASE_URL
REDIS_URL
OBJECT_STORAGE_ENDPOINT
OBJECT_STORAGE_BUCKET
OBJECT_STORAGE_ACCESS_KEY
OBJECT_STORAGE_SECRET_KEY
ARIA_PROVIDER
ARIA_MODEL
API_BASE_URL
```

Secrets shall never be committed to source control.

---

70. ENVIRONMENT FILE

The repository shall contain:

```
.env.example
```

The example shall document required configuration without containing production credentials.

---

71. DOCKER DEVELOPMENT STACK

The development environment shall be executable through:

```
docker-compose.yml
```

Initial services:

```
web
api
postgres
redis
worker-ai
worker-geometry
worker-validation
worker-drawing
worker-document
worker-render
object-storage
```

The local stack should allow the complete application to start from a clean environment.

---

72. DEVELOPMENT STARTUP

The target developer workflow shall become:

```
git clone
 ↓
configure .env
 ↓
docker compose up
 ↓
database migrations
 ↓
seed development data
 ↓
open OneDraft
```

The exact commands shall be documented in:

```
README.md
```

---

73. SEED SYSTEM

Development seed data shall create:

```
development organization
development user
residential project type
commercial project type
ARCH D template
wall assembly
door type
window type
test jurisdiction
golden project
```

All seed data shall be explicitly marked as development/test data.

---

74. SECURITY MODEL

Security shall follow:

```
authenticate
 ↓
authorize
 ↓
validate tenancy
 ↓
validate project access
 ↓
execute capability
```

A valid token alone shall not grant access to every project.

---

75. TENANCY MODEL

The organization is the primary tenant boundary.

The hierarchy is:

```
Organization
 ↓
Users / Memberships
 ↓
Projects
 ↓
Project Objects
```

Every project-scoped request shall resolve the organization relationship before execution.

---

76. API IDEMPOTENCY

Mutating external requests shall support:

```
Idempotency-Key
```

The API shall persist the request identity and resulting response.

Retries therefore cannot unintentionally create:

```
duplicate projects
duplicate commands
duplicate revisions
duplicate jobs
duplicate document packages
```

---

77. ASYNCHRONOUS COMMANDS

Commands that require substantial computation shall use:

```
API
 ↓
Command accepted
 ↓
Job created
 ↓
Worker execution
 ↓
Validation
```

```
 ↓
Revision finalization
```

The API shall return a Job_ID rather than blocking indefinitely.

---

78. SYNCHRONOUS COMMANDS

Simple operations may remain synchronous when they can complete safely within the API transaction boundary.

Examples:

```
create project
create space
change simple parameter
create relationship
record redline
```

The implementation shall not force asynchronous processing where it provides no architectural benefit.

---

79. EVENT-DRIVEN EXTENSION

The initial implementation may use explicit service calls.

The event model shall nevertheless establish future compatibility with:

```
event bus
message broker
distributed workers
external integrations
```

The audit event remains immutable regardless of future transport architecture.

---

80. DOCUMENT GENERATION PIPELINE

The document worker shall execute:

```
LOAD REVISION
 ↓
VERIFY MODEL VERSION
 ↓
VERIFY CODE MANIFEST
 ↓
VERIFY VALIDATION STATE
 ↓
LOAD DRAWINGS
```

```
 ↓
COMPOSE SHEETS
 ↓
RENDER
 ↓
GENERATE PDF
 ↓
GENERATE DXF
 ↓
STORE ARTIFACTS
 ↓
CALCULATE HASH
 ↓
CREATE DOCUMENT PACKAGE
```

---

81. FINALIZATION GATE

A document package shall not become a final customer-facing package until required validation conditions have been satisfied.

The finalization gate shall verify:

```
revision exists
model version exists
code manifest exists where required
validation completed
required validation conditions satisfied
drawings correspond to revision
documents correspond to revision
artifact hashes calculated
```

---

82. CUSTOMER ACCEPTANCE

Customer acceptance shall reference:

```
project
revision
model version
document package
code manifest
```

Acceptance does not modify historical project state.

It records the customer's acceptance of the identified package.

---

83. PROFESSIONAL REVIEW

Where professional review is required, the review record shall reference the exact revision under review.

The system shall preserve:

```
reviewer
professional role
scope
result
comments
review timestamp
```

---

84. APPROVAL TRACKING

Municipal or authority approval shall remain a separate project status domain.

OneDraft shall distinguish:

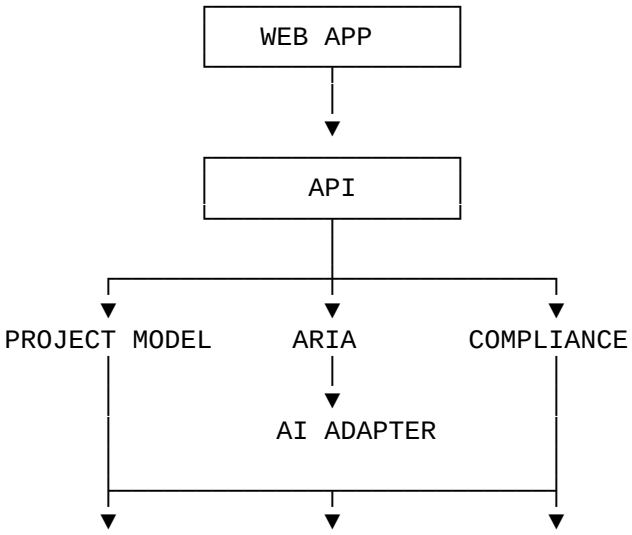
```
OneDraft validation
professional review
customer acceptance
authority submission
authority decision
```

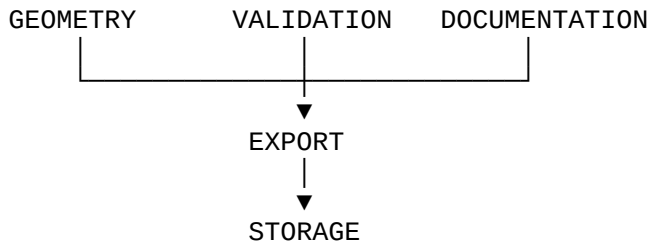
These are separate states and shall not be conflated.

---

85. SERVICE DEPENDENCY GRAPH

The initial dependency graph is:





---

86. SYSTEM DATA FLOW

The fundamental data flow is:

```
USER SPECIFICATION
 ↓
PROJECT REQUIREMENTS
 ↓
PROJECT MODEL
 ↓
ARIA INTERPRETATION
 ↓
MODEL COMMAND
 ↓
MODEL REVISION
 ↓
GEOMETRY
 ↓
COMPLIANCE
 ↓
VALIDATION
 ↓
DOCUMENTATION
 ↓
DRAWING SET
 ↓
DOCUMENT PACKAGE
 ↓
CUSTOMER ACCEPTANCE
```

---

87. MODEL-FIRST RULE

Every drawing-generation operation must originate from a model version.

No drawing subsystem may become an independent design-authority system.

This ensures:

```
model changes
 ↓
derived documentation changes
```

rather than:

```
drawing changes
  ↓
unknown model state
```

---

88. REDLINE-FIRST REVISION RULE

A redline shall become a model operation.

The workflow is:

```
REDLINE
  ↓
INTERPRETATION
  ↓
COMMAND
  ↓
MODEL CHANGE
  ↓
VALIDATION
  ↓
NEW REVISION
  ↓
DRAWING REGENERATION
```

A redline is therefore an instruction against the model rather than a permanent graphical patch.

---

89. CODE-FIRST VALIDATION RULE

Where jurisdictional requirements apply, OneDraft shall establish the applicable regulatory manifest before final documentation generation.

The intended sequence is:

```
PROJECT LOCATION
  ↓
JURISDICTION
  ↓
CODE SOURCES
  ↓
CODE MANIFEST
  ↓
APPLICABLE RULES
  ↓
PROJECT CONSTRAINTS
  ↓
MODEL
  ↓
VALIDATION
```

---

90. REPRODUCIBILITY

A historical package must be reproducible from its recorded provenance.

The system must preserve:

```
project ID
revision ID
model version ID
code manifest ID
validation run
drawing references
document package
artifact hashes
```

This provides the basis for historical reconstruction.

---

91. FAILURE ISOLATION

Failure of one computational subsystem shall not corrupt the authoritative model.

Examples:

```
geometry failure
→ model remains intact

drawing failure
→ model remains intact

PDF failure
→ model remains intact

AI provider failure
→ model remains intact

validation worker failure
→ model remains intact
```

The system shall prefer retryable derived-state failure over corruption of authoritative state.

---

92. AI PROVIDER FAILURE

If the underlying AI provider becomes unavailable:

```
Project Model
Database
Revisions
```

```
Existing Drawings
Existing Documents
Audit History
```

remain accessible.

The AI subsystem is therefore replaceable infrastructure rather than the database of record.

---

93. GEOMETRY PROVIDER FAILURE

Likewise, geometry computation shall be isolated.

Existing validated geometry remains identified by its:

```
geometry_version
geometry_hash
model_version_id
```

New geometry may be regenerated when the geometry subsystem becomes available.

---

94. DATABASE AS AUTHORITY

PostgreSQL remains authoritative for transactional state.

Redis is not authoritative.

Object storage is not authoritative.

AI provider state is not authoritative.

Browser state is not authoritative.

Generated PDF is not authoritative.

The Project Model persisted through PostgreSQL is authoritative.

---

95. IMPLEMENTATION PHASE ONE

Phase One shall establish:

```
repository
Docker environment
PostgreSQL
Redis
API
domain types
database migrations
```

```
OpenAPI contract
Project Model service
```

The result is the first executable backend.

---

96. IMPLEMENTATION PHASE TWO

Phase Two shall establish:

```
web application
authentication
project creation
project specification
model inspection
basic element editing
```

The result is the first usable OneDraft interface.

---

97. IMPLEMENTATION PHASE THREE

Phase Three shall establish:

```
Aria orchestration
structured commands
command validation
revision creation
constraint preservation
```

The result is the first model-aware Aria workflow.

---

98. IMPLEMENTATION PHASE FOUR

Phase Four shall establish:

```
geometry kernel
geometry workers
wall generation
door generation
window generation
space boundaries
basic 3D model
```

The result is the first computational building model.

---

99. IMPLEMENTATION PHASE FIVE

Phase Five shall establish:

```
views
plans
elevations
sections
dimensions
annotations
ARCH D sheets
DXF
PDF
```

The result is the first actual architectural documentation pipeline.

---

100. IMPLEMENTATION PHASE SIX

Phase Six shall establish:

```
code sources
code rules
code manifests
validation engine
validation results
```

The result is the first regulatory verification layer.

---

101. IMPLEMENTATION PHASE SEVEN

Phase Seven shall establish:

```
redlines
redline interpretation
redline resolution
affected drawing detection
drawing regeneration
```

The result is the first iterative design-review loop.

---

102. IMPLEMENTATION PHASE EIGHT

Phase Eight shall establish:

```
document packages
hashing
```

```
acceptance
professional review
approval tracking
complete audit history
```

The result is the first complete project lifecycle.

---

103. INITIAL EXECUTABLE ARTIFACTS

The immediate implementation package shall contain:

```
database/migrations/001_initial_schema.sql

docs/openapi/openapi.yaml

packages/domain/

packages/schemas/

services/project_model/project_model_service.py
```

These artifacts form the first executable OneDraft foundation.

---

104. INITIAL INFRASTRUCTURE ARTIFACTS

The implementation package shall also establish:

```
docker-compose.yml
.env.example
pyproject.toml
package.json
README.md
```

These files establish the reproducible development environment.

---

105. FIRST BUILD TARGET

The first successful build shall demonstrate:

```
Docker starts
 ↓
PostgreSQL starts
 ↓
Redis starts
```

```
 ↓
Migrations execute
 ↓
API starts
 ↓
OpenAPI loads
 ↓
Project can be created
 ↓
Project can be persisted
 ↓
Project can be retrieved
```

No CAD rendering is required for this first build target.

---

106. SECOND BUILD TARGET

The second build shall demonstrate:

```
Project
 ↓
Building
 ↓
Levels
 ↓
Spaces
 ↓
Elements
 ↓
Constraints
 ↓
Revision
 ↓
Model reconstruction
```

This proves the computational project model.

---

107. THIRD BUILD TARGET

The third build shall demonstrate:

```
User request
 ↓
Aria
 ↓
Command
 ↓
Constraint verification
 ↓
Model mutation
 ↓
```

```
Revision
  ↓
Affected objects
```

This proves the AI command boundary.

---

108. FOURTH BUILD TARGET

The fourth build shall demonstrate:

```
Model
  ↓
Geometry
  ↓
Plan
  ↓
Elevation
  ↓
Section
  ↓
ARCH D sheet
  ↓
PDF / DXF
```

This proves the CAD-AI documentation pipeline.

---

109. FIFTH BUILD TARGET

The fifth build shall demonstrate:

```
Project
  ↓
Jurisdiction
  ↓
Code Manifest
  ↓
Validation
  ↓
Validation Results
  ↓
Document Package
```

This proves the regulatory/documentation relationship.

---

110. SIXTH BUILD TARGET

The sixth build shall demonstrate:

```
Redline
 ↓
Aria
 ↓
Command
 ↓
Revision
 ↓
Validation
 ↓
Drawing regeneration
 ↓
New document package
```

This proves the iterative production workflow.

---

111. COMPLETE SYSTEM TARGET

The complete MVP shall execute:

```
SPECIFICATION
 ↓
PROJECT CREATION
 ↓
MODEL GENERATION
 ↓
CODE ANALYSIS
 ↓
MODEL VALIDATION
 ↓
DRAWING GENERATION
 ↓
CUSTOMER REDLINE
 ↓
ARIA REVISION
 ↓
REVALIDATION
 ↓
DOCUMENT GENERATION
 ↓
ACCEPTANCE
 ↓
HISTORICAL RECONSTRUCTION
```

without requiring direct database manipulation by the customer.

---

112. ENGINEERING PRINCIPLE

OneDraft is not an AI wrapper around a drawing generator.

It is a computational architectural information system in which:

```
THE MODEL IS PRIMARY
ARIA IS INTELLIGENCE
GEOMETRY IS COMPUTATION
COMPLIANCE IS VERIFICATION
DOCUMENTATION IS DERIVATION
VERSIONING IS PROVENANCE
REDLINES ARE MODEL OPERATIONS
VALIDATION IS A GATE
ACCEPTANCE IS A RECORDED STATE
```

---

113. STACK BOUNDARY

The implementation stack is therefore:

```
┌──────────────────────────────────────────────────┐
│                    ONEDRAFT                      │
├──────────────────────────────────────────────────┤
│ Web                                              │
│ Next.js / React / TypeScript                     │
├──────────────────────────────────────────────────┤
│ API                                              │
│ FastAPI / Python                                 │
├──────────────────────────────────────────────────┤
│ Domain                                           │
│ Project Model / Pydantic                         │
├──────────────────────────────────────────────────┤
│ Intelligence                                     │
│ Aria / Provider Adapter                          │
├──────────────────────────────────────────────────┤
│ Computation                                      │
│ Geometry / Compliance / Validation               │
├──────────────────────────────────────────────────┤
│ Documentation                                    │
│ Drawing / Rendering / Export                     │
├──────────────────────────────────────────────────┤
│ Async                                            │
│ Redis / Workers                                  │
├──────────────────────────────────────────────────┤
│ Persistence                                      │
│ PostgreSQL                                       │
├──────────────────────────────────────────────────┤
│ Artifacts                                        │
│ S3-Compatible Object Storage                     │
├──────────────────────────────────────────────────┤
│ Infrastructure                                   │
│ Docker                                           │
└──────────────────────────────────────────────────┘
```

---

114. NEXT STACK STATE

The OneDraft implementation architecture is now established as:

```
PRODUCT DEFINITION
        ↓
SYSTEM ARCHITECTURE
        ↓
DOMAIN MODEL
        ↓
DATABASE / API CONTRACT
        ↓
EXECUTABLE APPLICATION STACK
        ↓
IMPLEMENTATION
```

The next work is no longer architectural definition.

It is source-code construction.

---

115. NEXT IMPLEMENTATION ARTIFACT

The immediate next artifact shall be:

```
database/migrations/001_initial_schema.sql
```

followed by:

```
docs/openapi/openapi.yaml
```

then:

```
packages/domain/
packages/schemas/
services/project_model/project_model_service.py
```

The artifacts shall be generated against the established Database Schema & OpenAPI Contract v0.1 rather than independently redesigned.

---

116. IMPLEMENTATION HANDOFF

At this point OneDraft has an established:

```
product architecture
system architecture
technology stack
repository architecture
domain model
persistence model
API contract
AI boundary
geometry boundary
compliance boundary
validation boundary
```

```
documentation pipeline
revision system
redline system
acceptance system
audit system
testing architecture
deployment foundation
```

The architecture is therefore ready for executable construction.

---

117. VERSION 0.1 ENGINEERING STATE

ONEDRAFT EXECUTABLE APPLICATION STACK & IMPLEMENTATION ARCHITECTURE v0.1

STATUS: ESTABLISHED

The persistence/API layer has been converted into an executable-stack specification.

The system now proceeds from:

```
SPECIFICATION
```

to:

```
IMPLEMENTATION
```

with the Project Model remaining the single authoritative computational state.

---

118. NEXT ACTION

The next source artifact is:

```
001_initial_schema.sql
```

The implementation sequence is:

```
001_initial_schema.sql
        ↓
openapi.yaml
        ↓
domain_types.py
        ↓
project_model_service.py
        ↓
FastAPI application
        ↓
Docker development stack
        ↓
first integration test
```

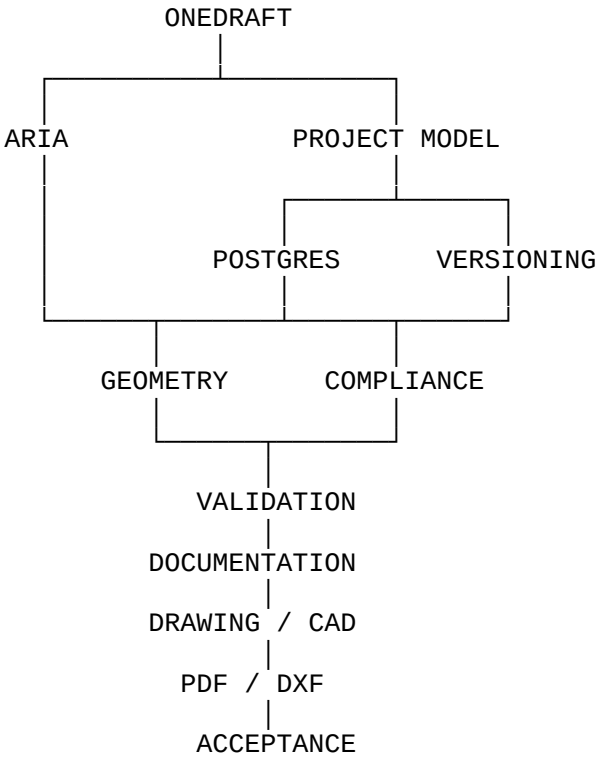
This establishes the first runnable OneDraft backend and creates the foundation upon which Aria, geometry, compliance, validation, drawing generation, and document production will operate.

---

119. FINAL STACK DECISION

OneDraft shall be implemented as a **persistent computational project-model platform**, not as a conversational interface with drawing outputs.

The definitive architecture is:



The architectural drawing remains the **derived representation**.

The Project Model remains the **computational source of truth**.

Aria remains the **intelligence layer operating through controlled domain capabilities**.

The database remains the **persistent transactional memory**.

The revision system remains the **historical state mechanism**.

The compliance manifest and validation system remain the **verification layer**.

The documentation engine remains the **production layer**.

This is the executable OneDraft stack.